Université de Strasbourg

Master Informatique 1ère année
Compilation - Projet 2025-26

Compilateur pour le langage **AFSTAT**

But du projet : Le but du projet est de réaliser un compilateur pour le langage AFSTAT, depuis le texte de haut niveau jusqu'à un code en C ou C++ compilable et exécutable.

1 Description du langage AFSTAT

Le langage AFSTAT (pour « Ask For Stat ») est un « langage métier » (en anglais *Domain Specific Language*, DSL), dont l'objectif est de faciliter, pour des analystes et statisticiens, l'interrogation de données afin de réaliser des calculs statistiques.

Sa syntaxe est déclarative, ce qui signifie que l'utilisateur décrit ce qu'il veut faire, et non comment le faire.

Un programme AFSTAT accède à des fichiers au format CSV, et débute classiquement par une ou plusieurs directives `source` qui indiquent au programme les chemins d'accès aux fichiers CSV à lire, une ou plusieurs définitions des structures des données accédées. Ensuite, le programme peut éventuellement contenir des opérations de jointure, ou des opérations de filtrage des données, ou des calculs dont les résultats constituent de nouvelles colonnes, puis enfin un bloc d'analyses à effectuer à partir des colonnes.

Il est possible d'intégrer des commentaires commençant par // dans le programme source.

1.1 Fichier d'entrée (`source`)

Cette directive indique au programme le chemin d'accès au fichier CSV à lire, et lui associe un nom logique. Sa syntaxe est : `source nom_source "chemin/vers/le/fichier.csv";`

1.2 Déclaration de schéma (`schema`)

La section `schema` définit la structure logique des données. Elle est cruciale pour que le code C/C++ généré sache comment lire et interpréter les colonnes du fichier CSV. Sa syntaxe est :

```
schema nom_du_schema {
    nom_colonne type_donnees;
    ...
}
```

Les types de données supportés sont :
- `integer` : Pour les nombres entiers (ex : âge, nombre de patients).
- `float` : Pour les nombres décimaux (ex : poids, température).
- `string` : Pour les chaînes de caractères (ex : nom, genre).

Exemple :

```
schema patient_data {
    id integer;
```

```
    age integer;
    gender string;
    weight float;
    height float;
}
```

Chaque fichier doit toujours avoir son propre schéma, en associant un schéma à une source de données. La syntaxe est : `associate nom_schema with nom_source;`

1.3 Définition des colonnes calculées (`compute`)

Le langage permet de créer de nouvelles colonnes basées sur des calculs utilisant les colonnes existantes. La syntaxe est : `compute nom_nouvelle_colonne = expression;`

Les opérateurs mathématiques supportés pour les expressions sont : `+, -, *, /`. Elles peuvent contenir des sous-expressions parenthésées et des constantes entières ou flottantes.

Exemple : Pour calculer l'IMC (Indice de Masse Corporelle) , on utilise la formule `poids / (taille * taille)` : `compute bmi = weight / (height * height);`

1.4 Spécification des analyses statistiques (`analyze`)

C'est dans cette section de code que l'utilisateur demande les calculs et visualisations souhaités. La syntaxe est :

```
analyze nom_du_schema {
    operation_statistique nom_colonne;
    ...
}
```

Les opérations statistiques supportées sont :
- `mean` : Calcule la moyenne.
- `median` : Calcule la médiane.
- `std_dev` : Calcule l'écart-type.
- `min` : Trouve la valeur minimale.
- `max` : Trouve la valeur maximale.
- `histogram` : Génère un histogramme de fréquence (pour les données catégorielles ou binnées).
- `summary` : Calcule une synthèse statistique complète (`min, max, mean, std_dev`) pour une colonne numérique.

Exemple :

```
analyze patient_data {
    summary age;
    mean weight;
    histogram gender;
}
```

On donne en figures 1, 2 et 3 des exemples de programme AFSTAT d'analyses de données médicales.

1.5 Croisement de plusieurs sources de données en AFSTAT

Un programme AFSTAT peut également accéder à plusieurs fichiers CSV pour effectuer des croisements de données. Plutôt que de n'avoir qu'une seule directive `source`, on en utilise alors plusieurs.

Chaque fichier doit toujours avoir son propre schéma, grâce à l'instruction `associate` :

```
schema nom_schema { ... }
associate nom_schema with nom_source;
```

On utilise une opération de jointure pour croiser les données. Elle combine des lignes de deux sources de données en fonction d'une condition de correspondance. La syntaxe est :

```
join nom_jointure = source1.colonne1 inner join source2.colonne2;
```

```
//  Analyse simple de données démographiques
// Ce programme lit un fichier de patients et calcule des statistiques de base
// sur leur âge et leur genre

// Spécification du fichier d'entrée
source patients "patients_medinfo.csv";

// Définition de la structure des données
schema patient_data {
    id integer;
    age integer;
    gender string;
}

associate patient_data with patients;

// Analyses à effectuer
analyze patient_data {
    mean age;
    histogram gender;
}
// Résultats attendus avec les fichiers donnés
// mean age: 51.5
// histogram gender:
// M: 15
// F: 15
```

FIGURE 1 – Exemple de code source en langage AFSTAT

Ceci crée une nouvelle source de données virtuelle qui contient les colonnes des deux sources pour les lignes où source1.colonne1 est égal à source2.colonne2.

À noter que plusieurs opération de jointure peuvent se succéder, qui elles-mêmes peuvent utiliser des noms de jointure, afin de réaliser une jointure de plus de deux fichiers.

On donne deux exemples de programmes utilisant des jointures figures 4 et 5.

1.6 Le filtrage des données

Le filtrage permet de cibler un sous-ensemble de données pour les analyses. L'instruction `filter` est placée après les déclarations de sources et les jointures sur lesquelles elle s'applique, et avant l'analyse qui la cible. Elle s'applique à une source de données (qu'elle soit une source initiale ou le résultat d'une jointure ou elle-même d'un filtrage) et produit une nouvelle source de données virtuelle. Sa syntaxe est : `filter id = source_des_données where condition`.

Les conditions sont de la forme `expression opérateur_de_comparaison expression`, où une expression peut utiliser des références à des colonnes de la source, les mêmes opérateurs arithmétiques que ceux utilisés pour les colonnes calculées, des constantes entières ou flottantes, et des parenthèses. Les opérateurs de comparaison possibles sont : `==, !=, <, >, <=, >=`.

On donne deux exemples de programmes qui utilisent l'instruction `filter` en figures 6 et 7.

2 Détection des erreurs

On désire apporter un soin particulier à la détection des erreurs de programmation, en affichant des messages les plus explicites possibles. Elles doivent toutes être détectées avant la compilation du programme C/C++ généré, dont la compilation devra se faire sans erreur. On utilisera le concept de promotion en flottant lors d'une comparaison ou d'un calcul entre opérandes entières et flottantes.

Les erreurs devant être détectées impérativement sont :

— Définition d'un schéma incohérent par rapport à la source de données associée. Le message d'erreur donnera explicitement les noms des colonnes inexistantes dans la source, ou des colonnes

```
// Calcul et analyse de l'IMC
// Ce programme calcule l'IMC avant d'en faire l'analyse

// Spécification du fichier d'entrée
source visits "visits_info.csv";

// Définition de la structure des données
schema health_data {
    id integer;
    weight float;
    height float;
}

associate health_data with visits;

// Définition d'une nouvelle colonne (bmi)
compute bmi = weight / (height * height);

// Analyses à effectuer
analyze health_data {
    summary weight; // Résumé du poids
    summary height; // Résumé de la taille
    summary bmi;    // Résumé de l'IMC
}
// Résultats attendus avec les fichiers donnés
// summary weight:
// min: 65.4
// max: 95.0
// mean: 78.8
// std_dev: 8.1
// summary height:
// min: 1.60
// max: 1.81
// mean: 1.70
// std_dev: 0.06
// summary bmi:
// min: 21.7
// max: 33.3
// mean: 26.5
// std_dev: 3.0
```

FIGURE 2 – Exemple de code source en langage AFSTAT

de types incompatibles entre schéma et source.
- Utilisation d'un **id** non défini.
- Jointure entre colonnes incompatibles, de types différents.
- Expression ou calcul avec des opérandes incompatibles.
- Analyses sur des colonnes incompatibles, comme par exemple la moyenne d'une colonne de chaînes de caractères.

2.1 Génération de code C/C++

Le code généré devra être en C ou C++, et compilable sans erreurs en utilisant le compilateur gcc ou g++.

3 Aspects pratiques et techniques

Le compilateur devra être écrit en C à l'aide des outils flex et bison. Vous devez donc concevoir la grammaire du langage AFSTAT.

Ce travail est à réaliser en équipe composée de quatre étudiant.e.s, et à rendre à la date indiquée

```
// Analyse avec des colonnes non utilisées
// Même si le fichier CSV a de nombreuses colonnes, le DSL permet de ne déclarer
// et d'analyser que celles qui vous intéressent, simplifiant ainsi le processus.
// Fichier CSV : "id,nom,age,ville,poids,taille,imc"
// L'utilisateur ne s'intéresse qu'à l'âge et à la taille.

source med "medical_records_full.csv";

schema data_subset {
    age integer;
    height float;
}

associate data_subset with med;

analyze data_subset {
    mean age;
    std_dev height;
}
// Résultats attendus avec les fichiers donnés
// mean age: 51.5
// std_dev height: 0.061
```

FIGURE 3 – Exemple de code source en langage AFSTAT

par vos enseignants en cours et sur Moodle. Une démonstration finale de votre compilateur sera faite durant la dernière séance de TP. Vous devrez rendre sur Moodle dans une archive :

- Le code source de votre projet complet dont la compilation devra se faire simplement par la commande « make ». Le nom par défaut du programme principal C/C++ produit doit être « afstat.c » ou « afstat.cpp ».
- Un document détaillant les capacités de votre compilateur, c'est-à-dire ce qu'il sait faire ou non. Soyez honnêtes, indiquez bien les points intéressants que vous souhaitez que le correcteur prenne en compte car il ne pourra sans doute pas tout voir dans le code.
- Un jeu de tests.

Votre compilateur devra fournir les options suivantes :

- `-version` devra indiquer les membres du projet.
- `-tos` devra afficher la table des symboles.
- `-o <name>` devra écrire le code résultat en C/C++ dans le fichier `name`.

4 Recommandations importantes

Écrire un compilateur est un projet conséquent, il doit donc impérativement être construit incrémentalement en validant chaque étape sur un plus petit langage et en ajoutant progressivement des fonctionnalités ou optimisations. Une démarche extrême et totalement contre-productive consiste à écrire la totalité du code du compilateur en une fois, puis de passer au débogage ! Le résultat de cette démarche serait très probablement nul, c'est-à-dire un compilateur qui ne fonctionne pas du tout ou alors qui reste très bogué.

Par conséquent, nous vous conseillons de développer tout d'abord un compilateur *fonctionnel* mais *limité* à la traduction d'instructions simples. À partir d'une telle version fonctionnelle, il vous sera plus aisé de la faire évoluer en intégrant telle ou telle fonctionnalité, ou en considérant des instructions plus complexes, ou en intégrant telle ou telle structure de contrôle. De plus, même si votre compilateur ne remplira finalement pas tous les objectifs, il sera néanmoins capable de générer des programmes corrects et qui «marchent» !

Il est impératif que les résultats des analyses s'affichent à l'écran, afin de pouvoir vérifier que vos programmes s'exécutent correctement et produisent des résultats justes !

```
// Définition des sources de données
source patients "data/patients.csv";
source records "data/records.csv";

// Définition et association des schémas
schema patients_schema {
    id integer;
    name string;
}
associate patients_schema with patients;

schema records_schema {
    patient_id integer;
    cholesterol float;
}
associate records_schema with records;

// Jointure des données sur l'identifiant du patient
join patient_cholesterol = patients.id inner join records.patient_id;

// Analyse sur les données combinées
analyze patient_cholesterol {
    summary cholesterol;
}
// Résultats attendus avec les fichiers donnés
// summary cholesterol:
// min: 75.6
// max: 270.6
// mean: 224.2
// std_dev: 25.1
```

FIGURE 4 – Exemple de code source en langage AFSTAT utilisant `join`

5 Précisions concernant la notation

- Si votre projet **ne compile pas, ou plante directement, ou ne génère aucun code C/C++ correct, la note 0 (zéro) sera appliquée** : l'évaluateur n'a absolument pas vocation à aller chercher ce qui pourrait éventuellement ressembler à quelque chose de correct dans votre code. Il faut que votre compilateur s'exécute et qu'il fasse quelque chose de correct, même si c'est peu.
- Si vous manquez de temps, préférez faire moins de choses mais en le faisant bien et de bout en bout : **même s'il est incomplet, votre compilateur doit pouvoir générer des programmes C/C++ compilables et exécutables**.
- Élaborez des tests car cela fait partie de votre travail et ce sera donc évalué.
- Faites les choses dans l'ordre et focalisez sur ce qui est demandé. L'évaluateur pourra tenir compte du travail fait en plus (par exemple des optimisations de code) seulement si ce qui a été demandé a été fait et bien fait.
- Une conception modulaire et lisible sera fortement appréciée.
- Si vous vous prenez au jeu et ajoutez des fonctionnalités à votre langage et votre compilateur (par exemple support de nouvelles jointures ou de nouvelles capacités statistiques), **faites-le seulement une fois que le contrat de base est rempli**, et intégrez ces capacités avec des exemples dans votre document et des tests dans votre projet.

```
// Définition des sources
source patients "data/patients.csv";
source visits "data/visits.csv";

// Définition et association des schémas
schema patient_data {
    patient_id integer;
    age integer;
    gender string;
}
associate patient_data with patients;

schema visit_data {
    patient_id integer;
    visit_date string;
    weight float;
    height float;
}
associate visit_data with visits;

// Jointure des données pour analyser le poids et la taille
join patient_visits = patients.patient_id inner join visits.patient_id;

// Calculer le BMI pour chaque visite
compute bmi = weight / (height * height);

// Analyse sur les données jointes
analyze patient_visits {
    mean bmi;
    histogram gender;
}
// Résultats attendus avec les fichiers donnés
// mean bmi: 26.5
// histogram gender:
// M: 15
// F: 15
```

FIGURE 5 – Exemple de code source en langage AFSTAT utilisant `join`

```
// Ce programme sélectionne uniquement les patients dont l'âge
// est supérieur à 50 ans avant d'analyser leurs données
source patients_data "data/patients.csv";
source records_data "data/records.csv";

schema patient_schema {
    id integer;
    age integer;
    gender string;
}
associate patient_schema with patients_data;

schema record_schema {
    patient_id integer;
    cholesterol float;
}
associate record_schema with records_data;

// Jointure des données
join all_data = patients_data.id inner join records_data.patient_id;

// Filtrage pour ne conserver que les patients de plus de 50 ans
filter old_patients = all_data where all_data.age > 50;

// Analyse sur le sous-ensemble de données filtré
analyze old_patients {
    mean cholesterol;
    histogram gender;
}
// Résultats attendus avec les fichiers donnés
// mean cholesterol: 236.4
// histogram gender:
// M: 9
// F: 9
```

FIGURE 6 – Exemple de code source en langage AFSTAT utilisant `filter`

```
// Filtrage sur une chaîne de caractères
source employees "data/employees.csv";

schema employee_schema {
    id integer;
    name string;
    department string;
    salary float;
}
associate employee_schema with employees;

// Filtrage pour ne conserver que les employés du département "R&D"
filter r_and_d_employees = employees where employees.department == "R&D";

// Analyse sur les salaires de ce groupe
analyze r_and_d_employees {
    summary salary;
}
// Résultats attendus avec les fichiers donnés
// summary salary:
// min: 65000
// max: 75000
// mean: 70000
// std_dev: 3000
```

FIGURE 7 – Exemple de code source en langage AFSTAT utilisant `filter`